Churn Prediction Project

Step 1: Perform simple EDA to Understand the Dataset

Problem Definition:

The objective is to **predict** whether users will make a **fifth** purchase based on their **four most recent purchases**. In addition, the purchase **probability** sh ould be estimated for each user.

Data:

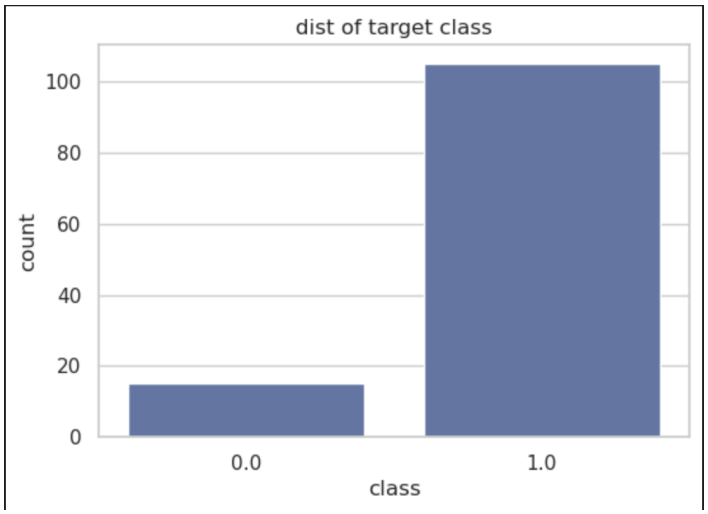
order_history:
It contains 800 rows. Each of the 200 users has exactly 4 recorded transactions.

labels:
Labels are available for 120 users (105 loyal and 15 churned). The remaining 80 users do not have labels, and their outcomes need to be predicted.

Features:
The values for *order_value* and *carton_count* have already been scaled. They are normalized and range between 0 and 1.

Identified Challenges:

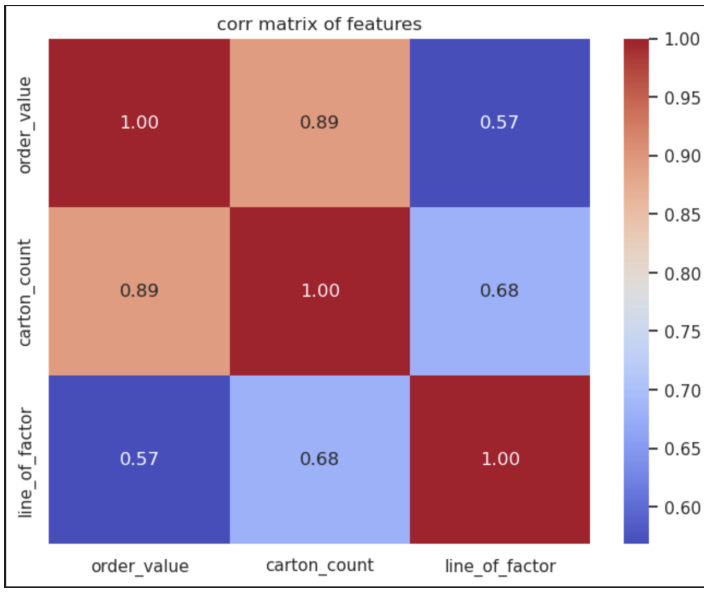
- **Imbalanced Data:** Since the number of loyal users (Class 1) is significantly higher than the number of churned users (Class 0), a model that always predicts Class 1 would achieve **high accuracy** but would not be practically useful. Therefore, evaluation metrics such as **F1-Score** or **AUC -ROC** should be used instead of relying solely on accuracy.
- **Limited Data Volume:** With only 120 labeled records available for training, the dataset is not sufficient for complex models, and there is a risk of overfitting.

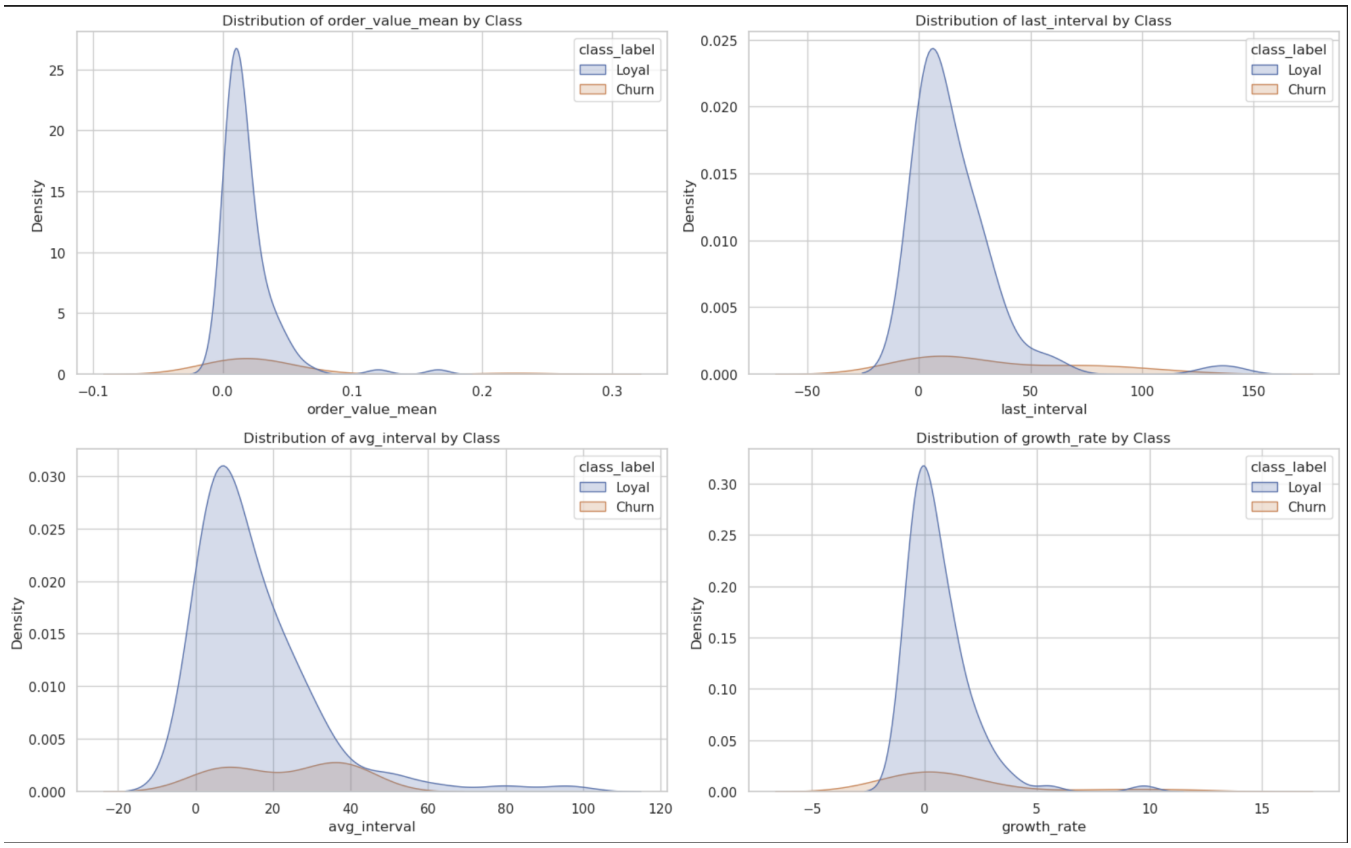


RFM:

In retail businesses, the **RFM (Recency, Frequency, Monetary)** framework is essential. Since *Frequency* is constant for all users (**4 purchases each**), the analysis should focus on **Recency** (time gap between purchases) and **Monetary** (purchase value).

High Correlation:
There is a strong correlation (0.89) between *order_value* and *carton_count*. This indicates that users who purchase **more cartons** naturally tend to have **hi gher invoice values**.





Step 2: Feature Engineering

Feature Selection is one of the most critical steps in data science. It combines business insight (domain knowledge) with statistical techniques (data science methods). Below, I explain the rationale behind the selected features and how to evaluate their quality. The selection was inspired by the standard **RFM framework (Recency, Frequency, Monetary)**, which is key in retail businesses. Since the number of purchases is fixed for all users (4 purchases), the focus is on the other two parameters. In this step, we consolidate the 4 transaction rows of each user into a **single feature row** that **summarizes their behavior over four purchases**. To better capture user behavior, features are grouped into three main categories:

1. Statistical Features (Aggregates):

- *Mean* and *standard deviation* of purchase value and item count.
  Rationale: The mean reflects the user's purchasing **power**, while the standard deviation indicates the **consistency** of their behavior.

2. Temporal Features (Recency & Pace):

- Time intervals between the first and last purchase, and specifically between the **3rd and 4th purchase**.
  Rationale: Increasing gaps between purchases may signal declining user interest.

3. Trend Features:

- Growth rate of purchase value from the first to the fourth transaction.
  Rationale: Users whose basket value is increasing over time are **more likely to return**.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 14 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   retail_id            200 non-null    object
 1   order_value_mean     200 non-null    float64
 2   order_value_sum      200 non-null    float64
 3   order_value_std      200 non-null    float64
 4   order_value_max      200 non-null    float64
 5   carton_count_mean    200 non-null    float64
 6   carton_count_std     200 non-null    float64
 7   line_of_factor_mean  200 non-null    float64
 8   line_of_factor_std   200 non-null    float64
 9   total_duration       200 non-null    float64
 10  avg_interval         200 non-null    float64
 11  last_interval        200 non-null    float64
 12  growth_rate          200 non-null    float64
 13  class                120 non-null    float64
dtypes: float64(13), object(1)
memory usage: 22.0+ KB
```

Time (Recency & Pace):

- In churn modeling, timing is the most critical factor.
- *last_interval:* A large gap between the 3rd and 4th purchase indicates that the user may be losing interest.
- *total_duration:* Shows how quickly the user completed their first 4 purchases.

Monetary & Quantity:

- *order_value_mean:* Reflects the user's purchasing power.
- *carton_count_std:* Captures variability in purchase volume. Users who consistently buy similar quantities tend to have more predictable—and usually more loyal—behavior.

Basket Diversity:

- *line_of_factor:* Measures the diversity of items purchased. Users who buy across more product categories typically become more attached to the platform (**sticky users**).

Step 3: Modeling & Comparison

At this stage, the goal is to identify the model with the strongest ability to distinguish between loyal and churned users. To ensure a fair comparison, the following standards were applied:

Algorithm Selection:

Five different algorithms were evaluated, covering a broad spectrum of modeling approaches:

Logistic Regression
Used as a **baseline model** to assess linear relationships between engineered features and churn probability.

Decision Tree
Selected to capture simple non-linear relationships while maintaining interpretability in decision-making.

Random Forest
Chosen due to its ensemble learning capability and high stability on small datasets through variance reduction (bagging).

XGBoost
Included as a state-of-the-art boosting algorithm, known for its strong performance in structured/tabular data problems.

MLP (Neural Network)
Tested to evaluate whether shallow neural architectures could extract additional non-linear patterns beyond tree-based models.

This diversified selection ensured that conclusions were not biased toward a single modeling philosophy.

Primary Evaluation Metric (ROC-AUC):
Due to the imbalanced dataset, **ROC-AUC** was used instead of accuracy. This metric better reflects the model's ability to separate the two classes.

Scaling:
Since linear models and neural networks are sensitive to feature scale, all features were standardized before training.

Stratified K-Fold Cross-Validation:
The dataset was split into **5 folds**, and each model was **evaluated 5 times**. This approach ensures that the results are **not driven by randomness**—especially important given the **limited data size**—and preserves the class distribution in each fold.

1.Why Use Cross-Validation?

Normally, we split data into **train and test sets** (e.g., 80/20). However, in this project, we only have **120 labeled samples**.

Problem with a Single Split:
If we divide the data just once, the test set contains only 24 users. Imagine if 3 or 4 **outlier users** end up in that small test set—model accuracy could fluctuate dramatically, making it unreliable.

Solution:
In **5-Fold Cross-Validation**, the dataset is split into 5 equal parts. The model is trained 5 times, each time using **one part as the test set** and the remaining 4 parts as the training set.

Result:
Every user appears **exactly once** in a test set. This provides a much more **stable and realistic estimate** of model performance, independent of which users happen to fall into the test set.

2.Why 5 Folds? (Why Not 10 or 3?)

Choosing the number of folds (k) is a **trade-off between bias and variance**:

If k=10:

- 90% of data is used for training (lower bias).
- But the test set is very small (only 12 users).
- **Main problem—class imbalance:** We have only 15 churned users (Class 0). With 10 folds, each test set would have just 1–2 churned users. If the model misclassifies even one of them, the accuracy for that fold would drop by 50%! This leads to **high variance** in results.

If k=3:

- Test sets are larger (40 users), but training sets shrink to 80 users.
- With only 80 samples, the model may **underfit** and fail to capture complex patterns (high bias).

Why 5 Folds is Optimal:

- Each test fold has about 24 users, with roughly 3 churned users per fold. This is **the minimum needed** to reliably compute metrics like Recall or AUC.
- Meanwhile, 96 users (80% of the data) remain for training, which is sufficient for tree-based models like Random Forest.

3.WhyStratified K-Fold:

- I used **Stratified K-Fold** to ensure that the **target class distribution** remains consistent across all training and evaluation folds. Given that we only have 15 churned samples, this technique prevents situations where a test fold contains **no churned users**, which would bias the evaluation results.

Initial Hyper Parameter Tuning Strategy: Manual

In the first phase, models were trained using manually selected hyperparameters (e.g., `max_depth=7`, `class_weight='balanced'`).

The purpose of this stage was rapid benchmarking and behavioral exploration

1. Logistic Regression

`class_weight='balanced'`:

- **Why?** Our dataset is highly imbalanced (105 loyal users vs. 15 churned users). This parameter tells the model to assign **higher weight and penalty** to mistakes on the minority class (churned users). Without it, the model would learn to predict everyone as “loyal” to achieve high apparent accuracy, which is misleading.

`random_state=42`:

- **Why?** Ensures that results are exactly the same on every run, providing **reproducibility**.

2. Decision Tree

`max_depth=7`:

- **Why?** The tree depth controls its complexity. With only 120 training samples, an unrestricted tree could memorize every data point (**overfitting**). A depth of 7 strikes a balance between learning meaningful patterns and avoiding noise.

`criterion='entropy'`:

- **Why?** This criterion measures the **purity** of nodes. Entropy is more sensitive to small probability changes than Gini, and in imbalanced datasets, it often produces **sharper decision boundaries**.

`min_samples_split=10`:

- **Why?** A node is only allowed to split if it contains at least 10 samples. This prevents the creation of very small, meaningless branches.

3. Random Forest (Winning Model)

`n_estimators=200`:

- **Why?** This is the number of trees in the forest. More trees make the model more stable and reduce variance. With our dataset size, 200 trees are sufficient for the **“wisdom of the crowd”** effect to stabilize predictions.

`max_depth=7`:

- **Why?** Similar to a single decision tree, this limits the complexity of each individual tree to prevent overfitting.

`max_features='sqrt'`:

- **Why?** At each split, only the square root of the total number of features is considered. This ensures that trees are diverse and less correlated with each other, which is a key principle of Random Forests.

4. XGBoost

`scale_pos_weight=7`:

- **Why?** This parameter is an advanced form of class weighting. Given the approximate 7:1 ratio (105 loyal vs. 15 churned), it tells the model to give **7 times more importance to the churned class**.

`learning_rate=0.05`:

- **Why?** A small learning rate ensures the model learns **slowly and steadily**. With limited data, a high learning rate (e.g., 0.3) could make the model fit too quickly to the training data, reducing its ability to generalize.

`max_depth=3`:

- **Why?** Boosting algorithms rely on **weak learners**. Shallow trees (depth 3) prevent overfitting and keep each step simple.

`subsample=0.8`:

- **Why?** Only 80% of the data is randomly used at each iteration. This prevents the model from depending too heavily on specific rows and improves robustness.

5. MLP (Neural Network)

`hidden_layer_sizes=(8, 4)`:

- **Why?** With only 120 training samples, large layers (e.g., 100 neurons) would be disastrous. We used a **very small, funnel-shaped structure** so the network focuses on capturing only the most general patterns.

`alpha=0.1`:

- **Why?** This is the **L2 regularization** parameter. A larger value keeps the network weights small, preventing extreme fluctuations and **overfitting**.

In all models, my primary focus was on **controlling overfitting**, given the very small training dataset (120 users). Additionally, by using **class weighting parameters**, I forced the models to carefully monitor the behavioral patterns of churned users (Class 0) despite the limited number of samples.

	Model	ROC-AUC	F1-Score
2	Random Forest	0.660317	0.933333
3	XGBoost	0.614286	0.928485
1	Decision Tree	0.603175	0.914746
0	Logistic Regression	0.603175	0.845578
4	Neural Network (MLP)	0.523810	0.922396

Due to the **limited dataset size**, bagging-based models (like Random Forest) demonstrated **greater stability** compared to boosting-based models.

Final Hyper Parameter Tuning Strategy: GridSearchCV

After identifying **Random Forest** and **XGBoost** as top-performing candidates, the workflow shifted from heuristic tuning to systematic optimization.

Why GridSearchCV?
To ensure that no optimal hyperparameter combinations (e.g., number of estimators, learning rate, tree depth) were overlooked.

To prevent biased evaluation, **Stratified 5-Fold Cross-Validation** was embedded within GridSearch. This ensured:

- Stable performance estimation across folds
- Preservation of class distribution in every split
- Reduced variance in AUC measurements
- Protection against random favorable splits

This approach increased confidence that performance improvements were genuine and not artifacts of data partitioning.

Final Results and Model Selection

After tuning, **Random Forest** and **XGBoost** achieved comparable and strong performance (AUC 0.66).

Random Forest
Demonstrated high robustness due to its **bagging mechanism**, making it resistant to noise and variance in small datasets.

XGBoost
Leveraged **regularization** and **sequential error correction**, showing strong capability in identifying minority-class (churn) patterns.

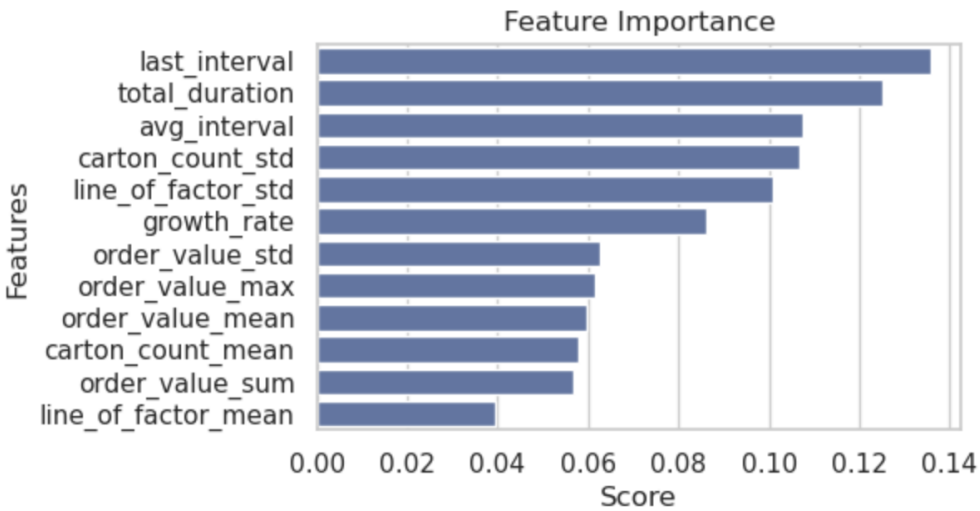
The outputs of both models were analyzed separately when predicting the 80 unlabeled users, providing complementary perspectives on churn risk.

	Model	Best AUC Score	Best Params
2	Random Forest	0.660317	{'class_weight': 'balanced_subsample', 'max_de...
3	XGBoost	0.660317	{'learning_rate': 0.1, 'max_depth': 3, 'n_esti...
4	MLP (Neural Net)	0.634921	{'activation': 'tanh', 'alpha': 0.0001, 'hidde...
0	Logistic Regression	0.622222	{'C': 10, 'class_weight': 'balanced', 'penalty...
1	Decision Tree	0.571429	{'class_weight': 'balanced', 'max_depth': 10, ...

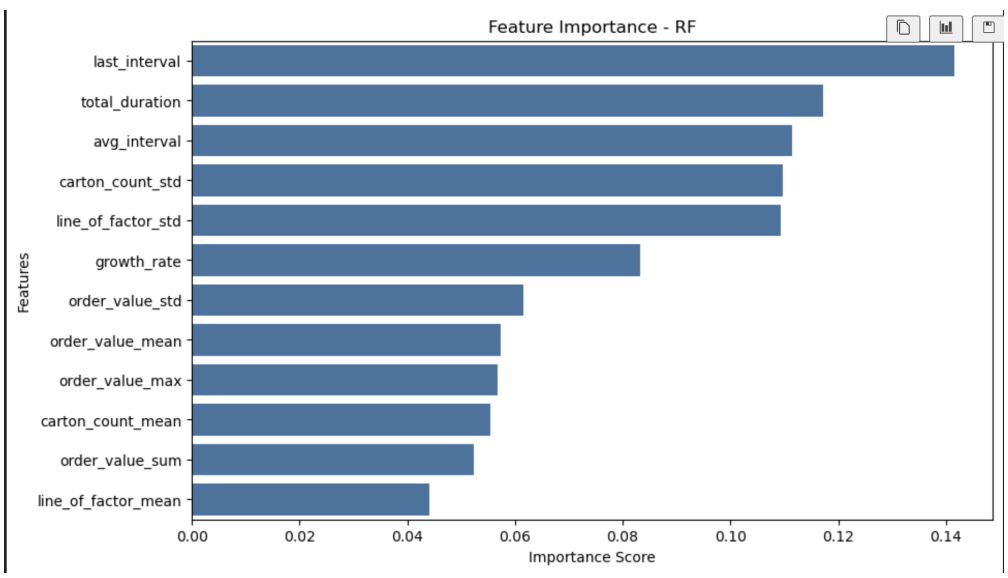
Step 4: Feature Importance

RandomForest - Manual Tuning:

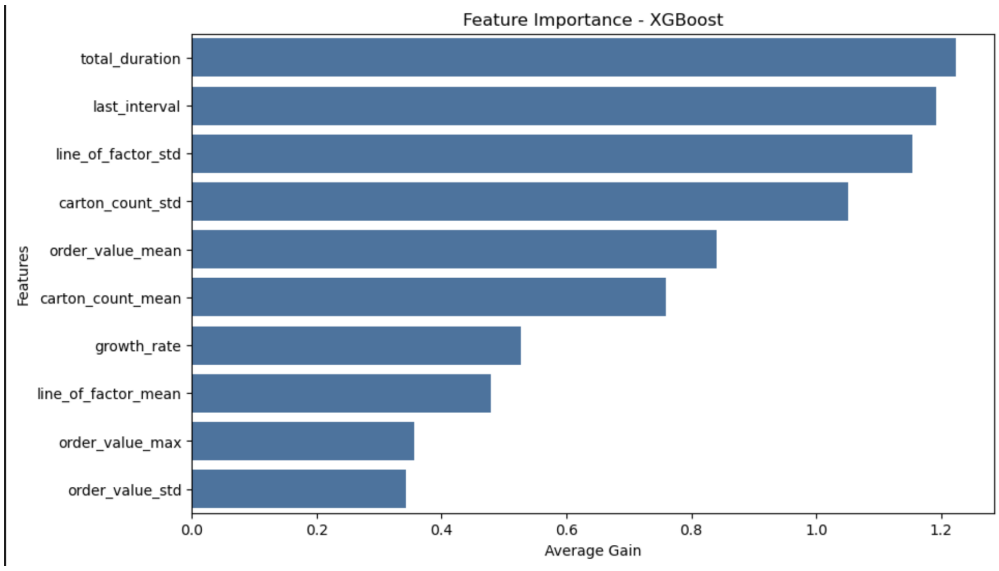
- **`last_interval` (13.5):** The most important feature. The time gap between the 3rd and 4th purchase is the strongest signal for predicting the 5th purchase. **Longer gaps increase the likelihood of churn.**
- **`total_duration` (12.5):** Indicates the overall pace of completing the first 4 purchases. Users who follow a consistent and faster rhythm tend to be **more loyal**.
- **`avg_interval` (10.7):** The average time between purchases. Helps the model assess whether a user's behavior is **"normal" or consistent**.
- **`carton_count_std` (10.6) & `line_of_factor_std` (10.0):** Both relate to behavioral stability. High variability in carton counts or item diversity per order signals **unpredictable behavior**, which in this dataset correlates with **higher churn probability**.



RandomForest - GridSearchCV Tuning:



XGBoost- GridSearchCV Tuning:



Step 6: Actionable Insights

1.Early Warning System
Since *last_interval* is the most critical factor, it's recommended that the product team implement a system where, if the time since a user's last purchase exceeds their **personal avg_interval** (e.g., 1.5×), an **automated SMS reminder** or **personalized offer** is sent. This helps prevent the user from "cooling off."

2.Consistency Campaign
Given the importance of **standard deviation features**, users with highly variable baskets are at higher risk of churn. The operations team can provide **stab le, subscription-like offers** to guide these users' purchasing behavior toward consistency.

3.Growth Recovery Plan

We saw that a negative growth rate (when the basket size in the fourth purchase is smaller than in the first purchase) is one of the signs of churn. For users whose **fourth purchase value has decreased** compared to the average of their first three purchases, run a **targeted upsell campaign**. Instead of giving a general discount, offer discounts on the categories the user used to buy before but are now missing from their basket. This can help bring the "basket depth" back to its normal level.

Step 7: Suggested Additional Data and Features

According to the task, if more data were available, the following features could further improve model accuracy:

- **Behavioral Data (App Event Logs):** Number of times the user opens the app or searches for products without completing a purchase. This provides insight into **purchase intent** before a transaction occurs.
- **Category Breadth:** The number of **different product categories** a user purchases from (e.g., dairy, protein, etc.). Greater diversity typically increases **platform stickiness**.
- **Support Interaction (Support Tickets):** Whether the user submitted complaints or support requests after the 3rd or 4th purchase. Poor early experiences are a major driver of churn.
- **Logistics Data:** The time gap between "order placement" and "product delivery," and the delivery delay rate + Adding **Delivery Delay** as a feature can greatly improve the prediction power of the model.
- **Payment Method:** Does the user pay in cash, or do they use credit (Credit/Wallet)?